



PROJECT MUSE®

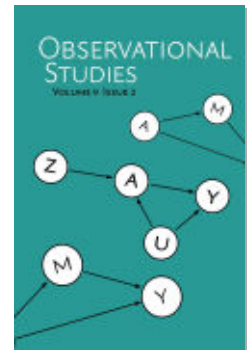
Doubly-Robust Inference in R using drtmle

David Benkeser, Nima S. Hejazi

Observational Studies, Volume 9, Issue 2, 2023, pp. 43-78 (Article)

Published by University of Pennsylvania Press

DOI: <https://doi.org/10.1353/obs.2023.0017>



➔ *For additional information about this article*

<https://muse.jhu.edu/article/883477>

Doubly-Robust Inference in R using drtmle

David Benkeser

benkeser@emory.edu

*Department of Biostatistics & Bioinformatics,
Rollins School of Public Health,
Emory University
Atlanta, GA 30322 USA*

Nima S. Hejazi

nhejazi@hsph.harvard.edu

*Department of Biostatistics,
T.H. Chan School of Public Health,
Harvard University
Boston, MA 02115 USA*

Abstract

Inverse probability of treatment weighted estimators and doubly robust estimators (including augmented inverse probability of treatment weight and targeted minimum loss estimators) are widely used in causal inference to estimate and draw inference about the average effect of a treatment. As an intermediate step, these estimators require estimation of key nuisance parameters, which are often regression functions. Typically, regressions are estimated using maximum likelihood and parametric models. Confidence intervals and p-values may be computed based on standard asymptotic results, such as the central limit theorem, the delta method, and the nonparametric bootstrap. However, in high-dimensional settings, maximum likelihood estimation often breaks down and standard procedures no longer yield correct inference. Instead, we may rely on adaptive estimators of nuisance parameters to construct flexible regression estimators. However, use of adaptive estimators poses a challenge for performing statistical inference about an estimated treatment effect. While doubly robust estimators facilitate inference when *all* relevant regression functions are consistently estimated, the same cannot be said when at least one nuisance estimator is inconsistent. `drtmle` implements doubly robust confidence intervals and hypothesis tests for targeted minimum loss estimates of the average treatment effect, in addition to several other recently proposed estimators of the average treatment effect.

Keywords: causal inference, machine learning, semiparametric estimation, treatment effects, multiple robustness, targeted minimum loss estimation

1. Introduction

In many fields, researchers are interested in assessing the population-level effect of a particular treatment on an outcome of interest. The treatment might correspond to a drug, a potentially harmful exposure, or a policy intervention. Often, the treatment cannot be randomized due to ethical or logistical reasons. This has sparked great interest in developing statistical methodology to estimate population-level effects from observational data. Estimation of these effects often requires accounting for putative confounding factors – that is, factors related to whether a participant receives treatment and to the participant’s outcome. In many settings, researchers measure many potential confounders, for example using ad-

ministrative databases or genetic sequencing technology. Due to the curse of dimensionality, common statistical estimation techniques are not feasible for such high-dimensional data without restrictive modeling assumptions. When these assumptions are violated, estimates of the population-level effect of a treatment can suffer from a large degree of bias.

This has led to increased interest in adopting adaptive estimation techniques from the machine learning literature to control for high-dimensional confounders when estimating treatment effects. However, facilitating proper statistical inference (i.e., confidence intervals and hypothesis tests) about estimates of effects based on adaptive estimators is challenging. In particular, the use of standard causal inference techniques, including both G-computation estimators (Robins 1986) and inverse probability of treatment weighted (IPTW) estimators (Horvitz and Thompson 1952; Robins, Hernan, and Brumback 2000) is challenging when utilizing adaptive estimators. In particular, with rare exceptions, if cross-validation is used to select tuning parameters for the nuisance parameters, estimates of the population-level effect will be too biased to achieve standard $n^{1/2}$ -rate asymptotics. Instead, *undersmoothing* must be used to select tuning parameters to control the bias (M. J. van der Laan, Benkeser, and Cai 2019; Ertefaie, Hejazi, and van der Laan 2022; Hejazi et al. 2022). However, this can prove quite challenging in practice, and few guidelines are available.

A more natural framework for integrating adaptive methods into nuisance regression estimation are doubly robust approaches, including augmented inverse probability of treatment weighted (AIPTW) estimation (Robins, Rotnitzky, and Zhao 1994) and targeted minimum loss (TML) estimation (M. J. van der Laan and Rubin 2006). The asymptotics of these estimators are characterized in part by the convergence rates of nuisance parameter estimates with respect to a proper norm. This fact makes the use of cross-validation more appealing for selection of tuning parameters for estimators of nuisance parameters, thereby providing a more straightforward means of obtaining an estimate of the effect of interest that enjoys standard $n^{1/2}$ asymptotics. Based on the efficient influence function, a key quantity in semiparametric theory, these asymptotic results also yield closed-form standard error estimates, which can serve as the basis for constructing confidence intervals and hypothesis tests.

As an intermediate step, the AIPTW and TML estimators require estimation of two key nuisance parameters: the probability of treatment as a function of confounders (the propensity score, hereafter referred to as the *PS*) and the average outcome as a function of confounders and treatment (the outcome regression, *OR*). In addition to their desirable inferential properties, AIPTW and TML estimators also enjoy a *double robustness* property. That is, the estimators are consistent for the population-level effect of interest if either of the PS or OR is consistently estimated. This property gives the AIPTW and TML estimators a natural appeal: Inconsistency in the estimation of one nuisance parameter may be mitigated by the consistent estimation of the other. However, recent work has shown that the double robustness property does not extend to the inferential properties of these estimators when adaptive estimators of the PS and/or OR are used (M. van der Laan 2014; D. Benkeser et al. 2017). Instead, valid inference requires consistent estimation of *both* the PS *and* the OR. M. van der Laan (2014) proposed new TML estimators that asymptotically attain a normal sampling distribution with estimable variance under consistent estimation of *either* the PS *or* the OR, paving the way for inference that is doubly-robust. D. Benkeser et al.

(2017) proposed modifications of these estimators with similar robustness properties and illustrated their practical performance.

The two proposed procedures that yield doubly-robust inference are quite involved, notably requiring iterative estimation of additional, complex nuisance parameters. The `drtmle` package was motivated by the need for an open source tool to implement these methods. The package implements the TML estimators of population-level effects proposed in M. van der Laan (2014) and D. Benkeser et al. (2017), as well as several extensions including cross-validated targeted minimum loss estimation (CV-TMLE). Additionally, `drtmle` implements an IPTW estimator that overcomes shortcomings of existing IPTW implementations with respect to statistical inference. In particular, the estimator allows an adaptive estimator of the PS to be used in construction of the IPTW estimator, while yielding valid statistical inference about population-level effects. Both the TML and IPTW estimators implemented in the `drtmle` package are capable of estimating population-level effects for discrete-valued treatments; moreover, the package provides convenient utilities for constructing confidence intervals and hypothesis tests about contrasts of the mean outcome under different levels of treatment. This manuscript explains some of the key concepts underlying doubly robust inference and illustrates usage of the `drtmle` package for the R language and environment for statistical computing (R Core Team 2022).

In tandem with growing interest in and usage of doubly robust estimation approaches, open source software tools implementing these methods have proliferated. Notably extant software packages include the `tmle` R package (Gruber and van der Laan 2012), which implements TML estimators of the effects of static single time point interventions; the `ltmle` R package (Lendle et al. 2017), which implements TML estimators of the effects of multiple time point interventions, subject to time-dependent confounding and censoring, as well as estimators of the parameters of a longitudinal working marginal structural model; the `tmle3` R package (Coyle 2021), which serves as a generalizable engine for implementing TML estimators within the `tlverse` ecosystem (M. J. van der Laan et al. 2023), designed to provide a common and extensible framework for TML estimation of the effects of static, dynamic, and stochastic single time point interventions; and the `doubleml` R and `Python` packages (Bach et al. 2021, 2022), which implement estimators of causal effects within the double machine learning framework (Chernozhukov et al. 2017, 2018), relying upon Neyman orthogonality and sample-splitting for robustness. From among this non-exhaustive list, `drtmle` stands out as, to our knowledge, the only implementation of statistical procedures designed to attain doubly robust inference (D. Benkeser et al. 2017), uniquely positioning `drtmle` as a robust tool for the estimation of treatment effects in observational data settings.

2. Definition and estimators of the average treatment effect

2.1 Parameters of interest

Suppose the observed data consist of n independent and identically distributed copies of the random variable $O = (W, A, Y) \sim P_0$. Here, W is a vector of baseline covariates, $A \in \{0, 1\}$ is a binary treatment, Y is an outcome, and P_0 is the true data-generating distribution. The parameters of interest are

$$\psi_0(a) = E_0\{\bar{Q}(a, W)\} = \int \bar{Q}_0(a, w) dQ_{W,0}(w), \text{ for } a = 0, 1, \quad (1)$$

where $\bar{Q}_0(a, w) = E_0(Y \mid A = a, W = w)$ is the so-called outcome regression and $Q_{W,0}(w) = Pr_0(W \leq w)$ is the distribution function of the covariates. The value $\psi_0(a)$ represents the marginal (i.e., population-level) treatment-specific average outcome, hereafter referred to as the *marginal mean* under treatment $A = a$. The marginal effect of the treatment on the outcome can be assessed by comparing marginal means under different treatment levels. For example, the average treatment effect is often defined as $\psi_0 := \psi_0(1) - \psi_0(0)$.

Under additional assumptions, these parameters identify the mean counterfactual outcomes under the different treatments. Specifically, we denote by $Y(a)$ a well-defined counterfactual outcome that would be observed under treatment a . If the conditional randomization assumption ($Y(a) \perp A \mid W$) and positivity conditions ($Pr_0\{Pr_0(A = a \mid W) > 0\} = 1$) hold, then $\psi(a)$ identifies the mean of $Y(a)$.

2.2 Estimators

The G-computation estimator of $\psi_0(a)$ may be motivated directly by (1). Suppose we have available an estimate of the OR, $\bar{Q}_n$, and an estimate of the distribution function of baseline covariates, $Q_{W,n}$. An estimator of $\psi_0(a)$ may be obtained by plugging these into (1), $\psi_n(a) = \int \bar{Q}_n(a, w) dQ_{W,n}(w)$. Typically, the empirical distribution function of covariates is used so that this estimator can be equivalently expressed

$$\psi_{n,\text{GCOMP}}(a) = \frac{1}{n} \sum_{i=1}^n \bar{Q}_n(a, W_i) . \quad (2)$$

Inference for GCOMP estimators based on parametric OR estimators may be facilitated through the nonparametric bootstrap. However, if adaptive approaches are used to estimate the OR, inference may not be available without further modification to the estimator (e.g., undersmoothing as discussed above).

An alternative estimator of the treatment-specific population-level mean may be obtained by noting that (2) can be equivalently written as

$$\begin{aligned} E_0\{\bar{Q}(a, W)\} &= E_0\{E_0(Y \mid A = a, W)\} \\ &= E_0\left\{\frac{I(A = a)}{Pr_0(A = a \mid W)} E_0(Y \mid A = a, W)\right\} \\ &= E_0\left\{\frac{I(A = a)}{Pr_0(A = a \mid W)} E_0(Y \mid A, W)\right\} \\ &= E_0\left\{\frac{I(A = a)}{Pr_0(A = a \mid W)} Y\right\} . \end{aligned} \quad (3)$$

We use $g_0(a \mid W) := Pr_0(A = a \mid W)$ to denote the propensity score. Suppose we had available g_n , an estimate of the PS. Equation (3) motivates the estimator

$$\psi_{n,\text{IPTW}}(a) := \frac{1}{n} \sum_{i=1}^n \frac{I(A_i = a)}{g_n(a \mid W_i)} Y_i ,$$

which is referred to as the inverse probability of treatment weighted (IPTW) estimator. Like GCOMP estimators, inference for IPTW estimators based on parametric PS estimators

may be facilitated through the nonparametric bootstrap. If adaptive approaches are used to estimate the PS, inference may not be available without further modification of the propensity score estimator, whether via targeting, as we discuss below, or undersmoothing (as in Ertefaie, Hejazi, and van der Laan (2022) and Hejazi et al. (2022)).

The GCOMP and IPTW estimators suffer from two important shortcomings. The first is a lack of robustness – that is, validity of GCOMP estimators relies on consistent estimation of the OR, while validity of the IPTW relies on consistent estimation of the PS. In practice, we may not know which of the OR or PS is simpler to consistently estimate and thus may not know which of the two estimators to prefer. Furthermore, if adaptive estimators (e.g., based on machine learning) are used to estimate the OR or PS, then the GCOMP and IPTW estimators generally lack a valid basis for asymptotic statistical inference.

The augmented IPTW (AIPTW) estimator

$$\psi_{n,\text{AIPTW}}(a) = \psi_{n,\text{IPTW}}(a) + \frac{1}{n} \sum_{i=1}^n \left\{ 1 - \frac{I(A_i = a)}{g_n(a | W_i)} \right\} \bar{Q}_n(a, W_i) ,$$

adds an augmentation term to the IPTW estimator. Equivalently, the estimator can be viewed as adding an augmentation to the GCOMP estimator

$$\psi_{n,\text{AIPTW}}(a) = \psi_{n,\text{GCOMP}}(a) + \frac{1}{n} \sum_{i=1}^n \frac{I(A_i = a)}{g_n(a | W_i)} \{Y_i - \bar{Q}_n(a, W_i)\} . \quad (4)$$

The AIPTW estimator overcomes the two limitations of the GCOMP and IPTW estimators. First, it exhibits *double robustness* in that it is consistent for $\psi_0(a)$ if either $\bar{Q}_n$ is consistent for the true OR *or* g_n is consistent for the true PS. Furthermore, if both the OR and PS are consistently estimated and regularity conditions hold, one can establish a normal limiting distribution for the AIPTW estimator. Closed-form estimators of the variance of this limiting distribution can be derived that may be used to construct Wald-style confidence intervals and hypothesis tests.

A more recent proposal to improve robustness and inferential properties of the GCOMP and IPTW estimators utilizes targeted minimum loss estimation (or TMLE). TMLE is a general methodology that may be used to modify an estimator of a regression function in a manner that ensures that the modified estimator satisfies user-selected equations. In the present problem, one can show that if an estimator of the OR $\bar{Q}_n^*$ satisfies

$$\frac{1}{n} \sum_{i=1}^n \frac{I(A_i = a)}{g_n(a | W_i)} \{Y_i - \bar{Q}_n^*(a, W_i)\} = 0 , \quad (5)$$

then the GCOMP estimator based on $\bar{Q}_n^*$ will have the same asymptotic properties as the AIPTW estimator. That is, the estimator will be doubly robust and, provided both the OR and PS are consistently estimated and regularity conditions are satisfied, will have a normal limiting distribution. TMLE provides a means of mapping an initial estimator of the OR into an estimator that satisfies (5). In addition to possessing the desirable asymptotic properties, TML estimators have been shown to outperform AIPTW estimators in finite samples (Porter et al. 2011). We refer readers to M. J. van der Laan and Rose (2011) and M. J. van der Laan et al. (2023) for further details about on the TMLE methodology.

3. Doubly robust inference

M. van der Laan (2014) demonstrated that the double robustness property of AIPTW and TML estimators does not extend to their normal limiting distribution if adaptive estimators of the OR and PS are used to construct the estimators. However, in settings with high-dimensional and continuous-valued covariates, adaptive estimation is likely necessary in order to obtain a consistent estimate of either the OR or PS, and thus may be necessary to obtain minimally biased estimates of the marginal means of interest. M. van der Laan (2014) derived modified TML estimators that are doubly robust not only with respect to consistency, but also with respect to asymptotic normality. Furthermore, this work provided closed-form, doubly robust variance estimators that may be used to construct doubly robust confidence intervals and hypothesis tests. D. Benkeser et al. (2017) proposed simplifications of M. van der Laan (2014)'s estimators and demonstrated the practical performance of these estimators via simulation experiments.

The key to the theory underlying these results is finding estimators of the OR and PS that simultaneously satisfy equation (5) in addition to two additional equations. These equations involve additional regression parameters that we refer to as the reduced outcome regression (R-OR) and the reduced propensity score (R-PS), defined respectively as

$$\bar{Q}_{r,0n}(a, w) := E_0\{Y - \bar{Q}_n(W) \mid A = a, g_n(W) = g_n(w)\} , \text{ and} \quad (6)$$

$$g_{r,0n}(a \mid w) := Pr_0\{A = a \mid \bar{Q}_n(W) = \bar{Q}_n(w), g_n(W) = g_n(w)\} . \quad (7)$$

In words, the R-OR is the regression of the residual from the outcome regression onto the estimated PS amongst observations with $A = a$, while the R-PS is the propensity for treatment $A = a$ given the estimated OR and PS. We note that a key feature of these reduced-dimension regressions is that they are low-dimensional regardless of the dimension of W and depend on neither the true OR nor the true PS. Thus, these regressions may be consistently estimated at reasonably fast rates irrespective of the dimension of W and irrespective of whether the OR or PS are consistently estimated. Based on these reduced-dimension regressions, M. van der Laan (2014) showed that if estimates of the OR $\bar{Q}_n^*$, PS g_n^* , R-OR $\bar{Q}_{r,n}^*$, and R-PS $g_{r,n}^*$ satisfied

$$\frac{1}{n} \sum_{i=1}^n \frac{I(A_i = a)}{g_n^*(a \mid W_i)} \{Y_i - \bar{Q}_n^*(a, W_i)\} = 0 , \quad (8)$$

$$\frac{1}{n} \sum_{i=1}^n \frac{\bar{Q}_{r,n}^*(a, W_i)}{g_n^*(a \mid W_i)} \{I(A_i = a) - g_n^*(a \mid W_i)\} = 0 , \text{ and} \quad (9)$$

$$\frac{1}{n} \sum_{i=1}^n \frac{I(A_i = a)}{g_{r,n}^*(a \mid W_i)} \left\{ \frac{g_{r,n}^*(a \mid W_i) - g_n^*(a \mid W_i)}{g_n^*(a \mid W_i)} \right\} \{Y_i - \bar{Q}_n^*(a, W_i)\} = 0 , \quad (10)$$

then, under regularity conditions described in Theorem 3 of M. van der Laan (2014), the GCOMP estimator based on $\bar{Q}_n^*$ would be doubly robust not only with respect to consistency, but also with respect to its limiting distribution. The author additionally proposed a TMLE algorithm to map initial estimates of the OR, PS, R-OR, and R-PS into estimates that satisfy these key equations. The regularity conditions required to achieve asymptotic

linearity include assumptions on the convergence rates of the OR, PS, R-OR, and R-PS to their true counterparts.

D. Benkeser et al. (2017) demonstrated alternative equations that can be satisfied to achieve this form of double-robustness. In particular, these authors formulated an alternative version of the R-PS,

$$g_{r,0,1}(a | w) := Pr_0\{A = a | \bar{Q}_n(W) = \bar{Q}_n(w)\} ,$$

which we refer to as R-PS1. They also introduced an additional regression of a weighted residual from the PS on the OR,

$$g_{r,0,2}(a | w) := E_0 \left\{ \frac{I(A = a) - g_n(a | W)}{g_n(a | W)} \mid \bar{Q}_n(W) = \bar{Q}_n(w) \right\} ,$$

which we refer to as R-PS2. The key feature of R-PS1 and R-PS2 is that they are both univariate regressions, while R-PS is a bivariate regression. D. Benkeser et al. (2017) suggested that this may make R-PS1 and R-PS2 easier to estimate consistently than R-PS. Further, they showed that if estimators of the OR, PS, and R-OR satisfy equations (8)–(9), and if, additionally, estimators of the OR, R-PS-1 $g_{r,n,1}^*$, and R-PS-2 $g_{r,n,2}^*$ satisfy

$$\frac{1}{n} \sum_{i=1}^n \frac{I(A_i = a)}{g_{r,n,1}^*(a | W_i)} g_{r,n,2}^*(a | W_i) \{Y_i - \bar{Q}_n^*(a, W_i)\} = 0 ,$$

then the GCOMP estimator based on this $\bar{Q}_n^*$ also enjoys a doubly robust limiting distribution. D. Benkeser et al. (2017) provided a TMLE algorithm that produced such estimates and provided closed-form, doubly robust variance estimates.

Remark: In the previous section, we introduced AIPTW and TMLE as two methods for constructing doubly robust (with respect to consistency) estimators of our parameter of interest. Theoretically, these two frameworks are closely related and, practically, the two estimators are generally seen as competitors for estimating marginal means. In equation (4), we were able to generate a doubly robust estimator by adding a term to the GCOMP estimator. This term is exactly the left-hand side of the key equation (5), the key equation that is solved by the standard TMLE. This is no coincidence – this term is crucial in studying the asymptotic properties of the GCOMP, AIPTW, and TML estimators (for discussion, see D. Benkeser et al. (2017), Section 2.2). Recall that a TML estimator achieves doubly-robust consistency by satisfying equation (5), while the AIPTW achieves doubly robust consistency by combining the GCOMP estimator with the left-hand side of this term. Now, we have argued that a TML estimator may achieve doubly robust limiting behavior if it additionally satisfies equations (9) and (10). It is thus sensible to wonder whether an AIPTW-style estimator could be constructed by combining the left-hand side of these two additional equations with the AIPTW estimator. D. Benkeser et al. (2017) showed that, surprisingly, this AIPTW-style estimator does not achieve the same doubly robust properties as the TML estimator. Thus, while AIPTW and TMLE are generally competitors for producing estimators doubly robust with respect to consistency, for doubly robust inference, TML estimators are preferable.

4. Implementation of doubly robust inference

The main workhorse of the `drtmle` package is the eponymous `drtmle()` function. This function estimates the treatment-specific marginal mean for user-specified levels of a discrete-valued treatment and computes a doubly robust covariance matrix for these estimates. The function implements either the estimators of M. van der Laan (2014) (if `reduction = "bivariate"`) or the improved estimators of D. Benkeser et al. (2017) (if `reduction = "univariate"`). The package includes utility functions for combining these estimates into estimates of average treatment effects, as well as other contrasts, as discussed below.

The main data arguments for `drtmle()` are `W`, `A`, `Y`, which, as above, represent the covariates, a discrete-valued treatment, and an outcome, respectively. Missing data are allowed in `A` and `Y`, as discussed below. Beyond the data arguments, it is necessary to specify how to estimate each of the OR, PS, R-OR, and R-PS (or R-PS1 and R-PS2). The package provides flexibility in how these regression parameters may be estimated. Once the user has specified the data arguments and how to estimate each regression parameter, the function proceeds as follows:

1. estimate the OR via the internal `estimateQ()` function;
2. estimate the PS via the internal `estimateG()` function;
3. estimate reduced-dimension regression via the internal `estimateQrn()` and `estimategrn()` functions;
4. iteratively modify initial estimates of the OR and PS via the `fluctuateQ()` and `fluctuateG()` functions until equations (8)–(10) are approximately solved;
5. compute the TML estimates and covariance estimates based on the final OR.

We refer interested readers to D. Benkeser et al. (2017) for a theoretical derivation of how the iterative modification of initial OR and PS is performed.

In addition to returning doubly robust parameter and covariance estimates, the `drtmle()` function returns estimates of the GCOMP, AIPTW, standard TMLE, and the modified AIPTW estimator discussed in the remark above. While doubly robust inference based on these estimators is not available, the estimators are provided for comparison. In the following subsections, we demonstrate the use of these methods by examining various options for making calls to `drtmle()`.

4.1 Estimating regressions

While the estimators computed by `drtmle()` are consistent and asymptotically normal under inconsistent estimation of either the OR or PS, it is nevertheless advisable to strive for consistent estimation of *both* the OR and PS. If both of the regression parameters are consistently estimated, then the estimators computed by `drtmle()` achieve the nonparametric efficiency bound (that is, the asymptotic variance of the efficient influence function), resulting in narrower confidence intervals and more powerful hypothesis tests. In order for the estimates of the OR and PS to have the greatest chance of consistency, the `drtmle()` function facilitates estimation using adaptive estimation techniques. In fact, there are three ways to estimate each of the regression parameters: generalized linear models, super learning (M. J. van der Laan, Polley, and Hubbard 2007; Polley et al. 2017), and a user-specified estimation technique. We demonstrate each of these approaches in turn on a simulated

data set in which the treatment is not randomized. The true parameter values of interest are $\psi_0(0) = 0.62$ and $\psi_0(1) = 0.72$.

```
set.seed(1234)
n <- 200
W <- data.frame(W1 = runif(n), W2 = rbinom(n, 1, 0.5))
A <- rbinom(n, 1, plogis(W$W1 + W$W2))
Y <- rbinom(n, 1, plogis(W$W1 + W$W2*A))
```

4.1.1 USING GENERALIZED LINEAR MODELS TO ESTIMATE REGRESSIONS

Generalized linear models are perhaps the most popular means of estimating regression functions. On account of this legacy, these estimators have been included as an option for estimating the regression parameters. The user specifies the right-hand side of a regression formula as a character string in the `glm_g`, `glm_Q`, `glm_Qr`, and `glm_gr` options to estimate the OR, PS, R-OR, and R-PS. These regressions are fit by calling the `glm()` function from R's base `stats` package (R Core Team 2022). Thus, the model specification should be able to be parsed as a valid formula object, as required by `glm()`. For the PS formula specified by `glm_g`, `colnames(W)` are available to use in the formula. For the OR (`glm_Q`) available names of variables to be used in the formula depends on the `stratify` option. If `stratify = FALSE`, a single OR is fit to estimate $\bar{Q}_0(A, W)$. In this case, the OR regression is fit by passing a `data.frame` that contains `W` and `A` into the `data` argument of `glm()`. Thus, both `colnames(W)` and `"A"` may appear in the formula specified in `glm_Q`. If `stratify = TRUE`, regressions are fit stratifying on `A` to separately estimate $\bar{Q}_0(1, W)$ and $\bar{Q}_0(0, W)$. In this case, the `glm_Q` formula may not include `"A"`.

To understand the variable names available for R-OR, recall from equation (6) that the nuisance parameter involves a regression onto the estimated propensity score. Thus, the formula that is input via `glm_Qr` should assume data with a single column, `"gn"`. Similarly, the R-PS1 and R-PS2 that are computed for the estimators of D. Benkeser et al. (2017) (the default estimators, computed whenever `reduction = "univariate"`) involve regressions onto the estimated outcome regression. Thus, the formula `glm_gr` should assume data with a single column named `"Qn"` if `reduction = "univariate"`. Instead, if the estimators of M. van der Laan (2014) are preferred (`reduction = "bivariate"`), the `glm_gr` formula specifies a model for (7) and assumes data with two columns, `"Qn"` and `"gn"`.

The code below illustrates how generalized linear models can be used to estimate the regression parameters. In this call to `drtmle()`, we specify `family = binomial()` so that logistic regression is used to estimate the OR. We illustrate non-stratified regression for both the univariate (default) and bivariate reductions below.

```
glm_fit_uni <- drtmle(W = W, A = A, Y = Y, family = binomial(),
  glm_g = "W1 + W2", glm_Q = "W1 + W2*A",
  glm_gr = "Qn", glm_Qr = "gn", stratify = FALSE,
  returnModels = TRUE)
glm_fit_uni
```

```
## $est
```

```
##
## 1 0.7111213
## 0 0.5383661
##
## $cov
##           1           0
## 1  1.741923e-03 -1.003719e-05
## 0 -1.003719e-05  4.081532e-03

glm_fit_biv <- drtmle(W = W, A = A, Y = Y, family = binomial(),
                    glm_g = "W1 + W2", glm_Q = "W1 + W2*A",
                    glm_gr = "Qn + gn", glm_Qr = "gn", stratify = FALSE,
                    returnModels = TRUE,
                    reduction = "bivariate")

glm_fit_biv

## $est
##
## 1 0.712154
## 0 0.537852
##
## $cov
##           1           0
## 1  1.721925e-03 -1.594085e-05
## 0 -1.594085e-05  4.009047e-03
```

The fit of the reduced-dimension regressions can be plotted using the post-hoc `plot.drtmle` method.

4.1.2 USING SUPER LEARNER TO ESTIMATE REGRESSIONS

Super learning is a loss-based ensemble machine learning technique (M. J. van der Laan, Polley, and Hubbard 2007) that generalizes stacking and stacked regression (Wolpert 1992; Breiman 1996). The user specifies many potential candidate regression algorithms, referred to as a *library* of candidate estimators, and the super learner algorithm estimates the convex combination of regression estimators that minimizes the V -fold cross-validated risk based on a user-selected loss function (Dudoit and van der Laan 2005). Oracle inequalities show that the super learner estimate performs essentially as well as the unknown best convex combination of the candidate estimators (van der Vaart, Dudoit, and van der Laan 2006). Thus, the methodology may be seen as an asymptotically optimal way of performing estimator selection. Practically, the super learner provides the best chance for obtaining a consistent estimator for both the OR and PS, and thereby an efficient estimator of the marginal means of interest.

Super learning is implemented in the R package **SuperLearner** (Polley et al. 2017) and `drtmle()` provides the option to utilize super learning for estimation of the OR, PS, and reduced-dimension regressions via the `SL_Q`, `SL_g`, `SL_Qr`, and `SL_gr` options. When calling

`drtmle()` either the generalized linear model options for regression modeling or the super learner options should be invoked. If both are called, the function will automatically opt to using the super learner option. The inputs to the super learner options are passed to the `SL.library` argument of the `SuperLearner()` function of the `SuperLearner` package. We refer readers to the documentation of the `SuperLearner()` function for information on how to properly specify a super learner library. In the code below, we illustrate a call to `drtmle()` with a relatively simple library. For the OR and PS, the library includes a main-terms generalized linear model (via the function `SL.glm` from `SuperLearner`) and an unadjusted generalized linear model (`SL.mean`). For the reduced-dimension regressions we use a super learner library with a main-terms generalized linear model (`SL.glm`) and adaptive regression splines (`SL.earth`, which utilizes the `earth` package (Milborrow et al. 2018)).

```
set.seed(1234)
sl_fit <- drtmle(W = W, A = A, Y = Y, family = binomial(),
               SL_g = c("SL.glm", "SL.mean"),
               SL_Q = c("SL.glm", "SL.mean"),
               SL_gr = c("SL.glm", "SL.earth"),
               SL_Qr = c("SL.glm", "SL.earth"),
               stratify = FALSE)
sl_fit

## $est
##
## 1 0.7121870
## 0 0.5368986
##
## $cov
##           1           0
## 1  1.718700e-03 -7.664583e-07
## 0 -7.664583e-07  4.154017e-03
```

The `drtmle` package also allows users to pass in their own functions for estimating regression parameters via the `SL_` options. These functions must be formatted properly for compatibility with `drtmle()`. The necessary formatting is identical to the formatting required for writing a `SuperLearner` wrapper (see `SuperLearner::write.SL.template()`). The `drtmle` package includes `SL.npreg`, which is an example of how a user can specify a function for estimating regression parameters. This functions fits a kernel regression estimator using the `np` package (Hayfield and Racine 2008). Below we show the source code for this function.

```
SL.npreg
```

```
## function (Y, X, newX, family = gaussian(), obsWeights = rep(1,
##      length(Y)), rangeThresh = 1e-07, ...)
```

```
## {
##   options(np.messages = FALSE)
##   if (abs(diff(range(Y))) <= rangeThresh) {
##     thisMod <- glm(Y ~ 1, data = X)
##   }
##   else {
##     bw <- np::npregbw(stats::as.formula(paste("Y ~", paste(names(X),
##       collapse = "+"))), data = X, ftol = 0.01, tol = 0.01,
##       remin = FALSE)
##     thisMod <- np::npreg(bw)
##   }
##   pred <- stats::predict(thisMod, newdata = newX)
##   fit <- list(object = thisMod)
##   class(fit) <- "SL.npreg"
##   out <- list(pred = pred, fit = fit)
##   return(out)
## }
## <bytecode: 0x7fe4e10db000>
## <environment: namespace:drtmle>
```

In the above code, we see that a user-specified function should have options `Y`, `X`, `newX`, `family`, `obsWeights`, ..., along with other options specific to the function. The option `Y` corresponds to the outcome of the regression and `X` to the variables onto which the outcome is regressed. The option `newX` should be of the same class and dimension as `X` and is meant to contain new values of variables at which to evaluate the regression fit. The `family` and `obsWeights` options are not strictly necessary (indeed, they are not used by `SL.npreg`), though these may be useful to include if one wishes to utilize the function as part of a super learner library. The `SL.npreg` function proceeds by checking whether there is sufficient variation in `Y` to justify fitting the kernel regression (as specified by option `rangeThresh`), and if so, fits a kernel regression with cross-validated bandwidth selection via the `npregbw()` function. The output is then formatted in a specific way so that `drtmle()` is able to retrieve the appropriate objects from the fitted regression. In particular the output should be a list with named objects `pred` and `fit`; the former including predictions made from the fitted regression on `newX`, the latter including any information that may be needed to predict new values based on the fitted object. A class is assigned to the `fit` object, so that an S3 `predict()` method may later be used to predict new values based on the fitted regression object. The user must also specify this `predict()` method. Use `drtmle:::predict.SL.npreg()` to view the `predict()` method for `SL.npreg`.

In the code below, we demonstrate a call to `drtmle()` with `SL.npreg` used to estimate each nuisance parameter. Here, we also illustrate the use of `stratify = TRUE`, which fits a separate kernel regression of `Y` onto `W` in observations with $A = 1$ and $A = 0$.

```
npreg_fit <- drtmle(W = W, A = A, Y = Y, family = binomial(),
  SL_g = "SL.npreg", SL_Q = "SL.npreg",
```

```
SL_gr = "SL.npreg", SL_Qr = "SL.npreg",
stratify = TRUE)
```

In the Appendix, we describe an additional means of using `drtmle()` with OR and PS estimates that have been obtained externally.

4.1.3 REDUCING DEPENDENCE ON RANDOM SEED

Some users may find it unsettling that when applying the super learner in real data analysis the results are dependent on the random seed that is set. While this is a natural artifact of cross-validation (sample-splitting) and adaptive estimation, one can average results over repeated super learner runs to reduce or remove this dependence – more repeats will lead to less dependence on the random seed. There are generally two approaches that can be used: (i) averaging on the scale of the nuisance parameters and (ii) averaging on the scale of the point estimates.

For (i), suppose that we have n_{SL} estimated super learners for the OR, $\bar{Q}_{n,j}, j = 1, \dots, n_{SL}$, and the PS $g_{n,j}, j = 1, \dots, n_{SL}$. We can take the average over these fits,

$$\bar{Q}_n = \frac{1}{n_{SL}} \sum_{j=1}^{n_{SL}} \bar{Q}_{n,j}$$

as our estimate of the OR (similarly for the PS). These averaged estimates are then used to compute the AIPTW or TMLE, as described above.

For (ii), suppose for example that we have n_{SL} TML (similarly, AIPTW) estimates $\psi_{n,j}(1)$ of $\psi_0(1)$, each obtained based on one (or more) super learners of the OR and/or PS. We can take the average over these point estimates,

$$\psi_n(1) = \frac{1}{n_{SL}} \sum_{j=1}^{n_{SL}} \psi_{n,j}(1)$$

as our estimate of $\psi_0(1)$.

This feature is implemented via the `n_SL` and `avg_over` options, where the former specifies the number of super learners to repeat and the latter specifies the scale(s) over which to average. For example, if `avg_over = "drtmle"` and `n_SL = 2`, then two point estimates are computed, each based on a single super learner, and averaged to obtain a final estimate. If instead `avg_over = "SL"` and `n_SL = 2`, then two super learners are fit and averaged before a final point estimate is obtained. Finally, if `avg_over = c("drtmle", "SL")` and `n_SL = 2` then averaging on *both* scales is performed. That is, the final estimate is the average of two point estimates, each obtained using OR and PS estimates that are averaged over two super learners.

Unsurprisingly, averaging over multiple super learners can add considerable computational expense to calls to `drtmle()`. The package is currently parallelized at the level of individual calls to `SuperLearner()`. With parallelization over enough cores, repeated super learning should not add too much to the computational burden relative to a single call to `SuperLearner()`.

4.2 Standard error estimation

This section contains some more advanced mathematical material and may be skipped over by users interested only in implementation. For simplicity, we focus this discussion on the standard TML and AIPTW estimators.

Reported standard error estimates are based on an estimate of the asymptotic variance of the estimators. This variance can be characterized by the (efficient) influence function of the estimators. For example, let $\psi_n(1)$ be an AIPTW estimate of $\psi_0(1)$. Under regularity conditions,

$$\psi_n(1) - \psi_0(1) = \frac{1}{n} \sum_{i=1}^n D^*(\bar{Q}_0, g_0, Q_{W,0})(O_i) + o_p(n^{-1/2}) ,$$

where

$$D^*(\bar{Q}_0, g_0, Q_{W,0})(O_i) = \frac{I(A_i = 1)}{g_0(1 | W_i)} \{Y_i - \bar{Q}_0(1, W_i)\} + \bar{Q}_0(1, W_i) - \int \bar{Q}_0(1, w) dQ_{W,0}(w) .$$

Thus, by the central limit theorem, $n^{1/2}\psi_n(1)$ converges in distribution to a normal distribution with mean $\psi_0(1)$ and variance

$$\sigma_0^2 = E_0([D^*(\bar{Q}_0, g_0, Q_{W,0})(O) - E_0\{D^*(\bar{Q}_0, g_0, Q_{W,0})(O)\}]^2) .$$

An estimate of σ_0^2 is naturally obtained by taking the empirical variance of $D_i = D^*(\bar{Q}_n, g_n, Q_{W,n})(O_i)$, that is,

$$\sigma_n^2 = \frac{1}{n} \sum_{i=1}^n \left\{ D_i - \frac{1}{n} \sum_{j=1}^n D_j \right\}^2 .$$

Thus, an estimate of the standard error of $\psi_n(1)$ is $\sigma_n/n^{1/2}$. Conveniently, this idea generalizes naturally to estimating the asymptotic covariance matrix of a vector $n^{1/2}(\psi_n(a) : a)$.

These ideas generalize immediately to the DR-TML estimator, where we can use the variance of the estimated influence function of that estimator. Unfortunately, the formulas for the influence functions more complex, but are available in Theorem 1 of D. Benkeser et al. (2017).

4.2.1 STANDARD ERRORS FOR TMLE

For TML estimators, it is standard practice to construct D_i using the *targeted* version of the OR (denoted by $\bar{Q}_n^*$ above). Recall that such estimates are obtained by modifying an initial OR estimate so as to ensure a particular (set of) equation(s) is (are) solved. However, simulations have shown that better standard error estimates for TMLE may be achieved by instead using the *initial* OR estimate in the computation of standard errors, as described in the preceding subsection. To this end, The `drtmle()` function includes the `targeted_se` option, which specifies whether initial nuisance parameter estimates are to be in computing the standard error (i.e., `targeted_se = FALSE`).

```

glm_fit_uni_nontargeted_se <-
  drtmle(W = W, A = A, Y = Y, family = binomial(),
    glm_g = "W1 + W2", glm_Q = "W1 + W2*A",
    glm_gr = "Qn", glm_Qr = "gn", stratify = FALSE,
    targeted_se = FALSE)
# compare to original call
list(
  glm_fit_uni$drtmle$cov, # based on targeted OR
  glm_fit_uni_nontargeted_se$drtmle$cov # untargeted OR
)

```

```

## [[1]]
##           [,1]           [,2]
## [1,] 1.741923e-03 -1.003719e-05
## [2,] -1.003719e-05  4.081532e-03
##
## [[2]]
##           [,1]           [,2]
## [1,] 1.745301e-03 -9.407578e-06
## [2,] -9.407578e-06  4.053834e-03

```

4.2.2 CROSS-VALIDATED STANDARD ERRORS

Simulation studies have revealed that when employing machine learning to estimate the OR and/or PS, the estimated standard errors are often *too small*, in small to moderate-sized samples. Problematically, this leads directly to under-coverage of confidence intervals and possibly inflated Type-I error rates of hypothesis tests based on these standard error estimates. This should not be too surprising, as the phenomenon is related to the form of the standard error estimates. In particular, such estimates include a term that resembles a (weighted) mean squared error (MSE) of the OR. Since machine learning algorithms often set out to minimize empirical MSE as part of their training process, it follows that the empirical MSE often ends up biased towards zero relative to the true MSE. To more accurately estimate the MSE of a machine learning algorithm, cross-validation can be used. The same sample-splitting idea can be used to obtain more conservative standard error estimates.

Suppose we use V -fold cross-validation to obtain V training-sample-specific estimates of the OR, $\bar{Q}_{n,v}$, $v = 1, \dots, V$ and the PS, $g_{n,v}$, $v = 1, \dots, V$. For $i = 1, \dots, n$, let $\bar{Q}_{n,i}^{cv}(1)$ denote the estimate of $\bar{Q}_0(a, W_i)$ obtained when observation i was in the validation fold. That is, $\bar{Q}_{n,i}^{cv}$ is an out-of-sample prediction of Y_i . We define $g_{n,i}^{cv}$ similarly. Using these estimates, we can define

$$D_i^{cv} = \frac{I(A_i = 1)}{g_{n,i}^{cv}} \{Y_i - \bar{Q}_{n,i}^{cv}(1)\} + \bar{Q}_{n,i}^{cv}(1) - \frac{1}{n} \sum_{i=1}^n \bar{Q}_{n,i}^{cv}(1)$$

and compute an estimate of σ_0^2 using

$$\sigma_{n,cv}^2 = \frac{1}{n} \sum_{i=1}^n \left\{ D_i^{cv} - \frac{1}{n} \sum_{j=1}^n D_j^{cv} \right\}^2 .$$

In `drtmle`, cross-validated standard error estimates can be obtained by setting `se_cv = "full"` and `se_cvFolds` to a number greater than one. For example,

```
glm_fit_uni_cv <- drtmle(W = W, A = A, Y = Y, family = binomial(),
  glm_g = "W1 + W2", glm_Q = "W1 + W2*A",
  glm_gr = "Qn", glm_Qr = "gn",
  se_cv = "full", se_cvFolds = 2, stratify = FALSE,
  targeted_se = FALSE) # needed for this to work
# compare variance to previously obtained
list(
  `No CV` = glm_fit_uni$drtmle$cov,
  `CV` = glm_fit_uni_cv$drtmle$cov
)
```

```
## $'No CV'
##           [,1]           [,2]
## [1,] 1.741923e-03 -1.003719e-05
## [2,] -1.003719e-05  4.081532e-03
##
## $CV
##           [,1]           [,2]
## [1,] 0.0024819103 0.0003380971
## [2,] 0.0003380971 0.0066362069
```

4.2.3 PARTIALLY CROSS-VALIDATED STANDARD ERRORS

Obtaining fully cross-validated standard errors can be fairly time-intensive when super learner is used to estimate the OR and PS. In `drtmle`, we give an option to compute standard error estimates that are *partially cross-validated* and do not require any additional fitting beyond a standard super learner. Here, we describe briefly how these estimates are obtained and show calls to `drtmle()` for obtaining such estimates.

Suppose we have K candidate regressions in our super learner library for both the OR and PS (in practice, the number could be different for the OR and for the PS). Recall that a super learner estimate of, for example, $\bar{Q}_0(a, w)$ is obtained as a weighted combination of estimates from each candidate regression. Let $\bar{Q}_{n,k}$ denote the estimate of $\bar{Q}_0$ obtained by training algorithm k using the full data set. The super learner estimate of $\bar{Q}_0(a, w)$ is

$$\bar{Q}_{n,SL}(a, w) = \sum_{k=1}^K \alpha_{n,k} \bar{Q}_{n,k}(a, w) ,$$

where $\alpha_n = (\alpha_{n,1}, \dots, \alpha_{n,K})$ is a weight vector that is selected by minimizing a cross-validated risk criterion. That is, each of the K algorithms is trained according to a V -fold cross-validation scheme, resulting in $V + 1$ fits of each algorithm (one in each of V training folds, plus one obtained by fitting the algorithm using the full data). Let $\bar{Q}_{n,k,v}$ denote the k^{th} algorithm trained using the v^{th} training fold. For $i = 1, \dots, n$, let $\bar{Q}_{n,k,i}^{cv}(1)$ denote the estimate of $\bar{Q}_0(1, W_i)$ implied by the fitted algorithm $\bar{Q}_{n,k,v}$, which was obtained when observation i was in the validation set. That is, for observations with $A_i = 1$, $\bar{Q}_{n,k,i}^{cv}(1)$ is an out-of-sample prediction of Y_i based on the k^{th} algorithm. Now, we can construct a *partially cross-validated* super learner prediction as follows. For $i = 1, \dots, n$ let

$$\bar{Q}_{n,SL,i}^{cv}(1) = \sum_{k=1}^K \alpha_{n,k} \bar{Q}_{n,k,i}^{cv}(1)$$

be the estimate obtained by ensembling the training-set-specific algorithm fits based on α_n . We can use these estimates to produce our *partially cross-validated* standard error estimate as follows. First, define

$$D_i^{pcv} = \frac{I(A_i = 1)}{g_{n,SL,i}^{cv}} \{Y_i - \bar{Q}_{n,SL,i}^{cv}(1)\} + \bar{Q}_{n,SL,i}^{cv}(1) - \frac{1}{n} \sum_{i=1}^n \bar{Q}_{n,SL,i}^{cv}(1),$$

then, compute an estimate of σ_0^2 using

$$\sigma_{n,pcv}^2 = \frac{1}{n} \sum_{i=1}^n \left\{ D_i^{pcv} - \frac{1}{n} \sum_{j=1}^n D_j^{pcv} \right\}^2.$$

Note that is not a properly cross-validated estimate, since observation i is used to compute the weight vector α_n . However, such estimates can be obtained based on the output of a single round of super learning, thereby providing improved standard error estimates at minimal additional computational cost.

Below, we demonstrate how to obtain such estimates using `drtmle()`.

```
set.seed(1234)
sl_fit_pcv <- drtmle(W = W, A = A, Y = Y, family = binomial(),
  SL_g = c("SL.glm", "SL.mean"),
  SL_Q = c("SL.glm", "SL.mean"),
  SL_gr = c("SL.glm", "SL.earth"),
  SL_Qr = c("SL.glm", "SL.earth"),
  stratify = FALSE,
  se_cv = "partial")
# compare estimates
# pt estimates should be same
# variance should be larger for pcv
list(
  `No-CV` = sl_fit$drtmle$cov, # no cv
  `Partial CV` = sl_fit_pcv$drtmle$cov # partial cv
)
```

```
## $'No-CV'
##           [,1]           [,2]
## [1,] 1.718700e-03 -7.664583e-07
## [2,] -7.664583e-07 4.154017e-03
##
## $'Partial CV'
##           [,1]           [,2]
## [1,] 1.813255e-03 -1.183808e-05
## [2,] -1.183808e-05 4.300791e-03
```

4.3 Confidence intervals

The `drtmle` package contains methods that compute Wald-style doubly robust confidence intervals. The `ci()` method computes confidence intervals about the estimated marginal means in a fitted object of S3 class "`drtmle`", in addition to contrasts between those means. By default, the method gives confidence intervals about each mean.

```
ci(npreg_fit)
```

```
## $drtmle
##      est    cil    ciu
## 1 0.716 0.636 0.796
## 0 0.534 0.411 0.658
```

Alternatively the `contrast` option allows users to specify a linear combination of marginal means, which can be used to estimate the average treatment effect, as we show below.

```
ci(npreg_fit, contrast = c(1, -1))
```

```
## $drtmle
##              est    cil    ciu
## E[Y(1)]-E[Y(0)] 0.182 0.034 0.329
```

The `contrast` option also accepts as input a list of functions to be used to compute a Wald-style confidence interval for a user-specified contrast. For example, we might be interested in the risk ratio $\psi_0(1)/\psi_0(0)$. When constructing Wald-style confidence intervals for ratios, it is common to compute the interval on the logarithmic scale and subsequently back-transform. That is, the confidence interval takes the form

$$\exp \left[\log \left\{ \frac{\psi_n(1)}{\psi_n(0)} \right\} \pm z_{1-\alpha/2} \sigma_n^{\log} \right],$$

where $\sigma_n^{\log}$ is the estimated standard error on the logarithmic scale. By the delta method, an estimate of this standard error is given by

$$\sigma_n^{\log} = \nabla g(\psi_n)^T \Sigma_n \nabla g(\psi_n),$$

where Σ_n is the doubly robust covariance matrix estimate output from `drtmle()` and $\nabla g(\psi)$ is the gradient of $\log\{\psi(1)/\psi(0)\}$. This confidence interval may be obtained as follows.

```

riskRatio <- list(f = function(eff){ log(eff) },
                 f_inv = function(eff){ exp(eff) },
                 h = function(est){ est[1] / est[2] },
                 fh_grad = function(est){ c(1 / est[1], -1 / est[2]) })
ci(npreg_fit, contrast = riskRatio)

## $drtmle
##               est    cil    ciu
## user contrast 1.34 1.036 1.733

```

More generally, this manner of inputting the `contrast` option to the `ci()` method can be used to compute confidence intervals of the form

$$f^{-1}\{f(h(\psi_n)) \pm z_{1-\alpha/2} \nabla f(h(\psi_n))^T \Sigma_n \nabla f(h(\psi_n))\},$$

where f (`contrast$f()`) is the transformation of the confidence interval, f^{-1} (`contrast$f_inv()`) is the back-transformation of the interval, h (`contrast$h()`) is the contrast of the marginal means, and $\nabla f(h(\psi_n))$ (`contrast$fh_grad()`) defines the gradient of the transformed contrast at the estimated marginal means.

We note that this method of computing confidence intervals can also be used to obtain a transformed confidence interval for a particular marginal mean. For example, recall that Y is binary in our running example. Thus, we may wish to enforce that our confidence interval be computed on a scale ensuring that the estimated confidence limits fall within the unit interval $[0, 1]$. To that end, we can utilize an interval of the form:

$$\text{expit} \left[\log \left\{ \frac{\psi_n(1)}{1 - \psi_n(1)} \right\} \pm z_{1-\alpha/2} \sigma_n^{\text{logit}} \right].$$

This confidence interval may be implemented as follows.

```

logitMean <- list(f = function(eff){ qlogis(eff) },
                 f_inv = function(eff){ plogis(eff) },
                 h = function(est){ est[1] },
                 fh_grad = function(est){ c(1 / (est[1] - est[1]^2), 0) })
ci(npreg_fit, contrast = logitMean)

## $drtmle
##               est    cil    ciu
## user contrast 0.716 0.629 0.789

```

4.4 Hypothesis tests

Doubly robust Wald-style hypothesis tests may be implemented using the `wald.test()` method. As with confidence intervals, there are three options for testing. First, we may test the null hypothesis that the marginal means equal a fixed value (or values). Below, we test the null hypothesis that $\psi_0(1) = 0.5$ and that $\psi_0(0) = 0.6$ by inputting a vector to the null option.

```
wald.test(npreg_fit, null = c(0.5, 0.6))
```

```
## $drtmle
##               zstat  pval
## H0: E[Y(1)]=0.5  5.276 0.000
## H0: E[Y(0)]=0.6 -1.044 0.297
```

As with the `ci()` method, the `wald.test()` method allows users to test linear combinations of marginal means by inputting a vector of ones and negative ones in the `contrast` option. We can use this to test the null hypothesis of no average treatment effect or to test the null hypothesis that the average treatment effect equals a given reference value, e.g., 0.05, by specifying the `null` option.

```
wald.test(npreg_fit, contrast = c(1, -1))
```

```
## $drtmle
##               zstat  pval
## H0:E[Y(1)]-E[Y(0)]=0 2.414 0.016
```

```
wald.test(npreg_fit, contrast = c(1, -1), null = 0.05)
```

```
## $drtmle
##               zstat  pval
## H0:E[Y(1)]-E[Y(0)]=0.05  1.75 0.08
```

The `wald.test()` method also allows for user-specified contrasts to be tested using syntax similar to the `ci()` method. The only difference syntactically is that it is not strictly necessary to specify `f_inv()`, as the hypothesis test is performed on the transformed scale. That is, we can generally test the null hypothesis that $f(h(\psi_0))$ equals f_0 (the function f applied to the value passed to `null`) using the Wald statistic

$$Z_n := \frac{\{f(h(\psi_n)) - f_0\}}{\nabla f(h(\psi_n))^T \Sigma_n \nabla f(h(\psi_n))} .$$

We can use the `riskRatio` contrast defined previously to test the null hypothesis that $\psi_0(1)/\psi_0(0) = 1$.

```
wald.test(npreg_fit, contrast = riskRatio, null = 1)
```

```
## $drtmle
##               zstat  pval
## H0: user contrast = 1 2.231 0.026
```

4.5 Missing data

The `drtmle()` function supports missing data in **A** and **Y**. Such missing data are common in practice and, if ignored, can bias estimates of the marginal means of interest. The estimators previously discussed can be modified to allow such missingness processes to depend on observed covariates. That is, if we define Δ_A and Δ_Y as indicators that A and Y are observed, respectively, then we can still perform valid inference on $E[Y(a)]$ under the assumptions that (i) $Y(a) \perp \Delta_A \mid W$, (ii) $Y(a) \perp A \mid \Delta_A = 1, W$, and (iii) $Y(a) \perp \Delta_Y \mid \Delta_A = 1, A = a, W$. In this case, we can redefine the OR to be $\bar{Q}_0(a, w) := E_0(\Delta_Y Y \mid \Delta_A A = a, \Delta_A = 1, \Delta_Y = 1, W = w)$ and similarly redefine the PS to be

$$\begin{aligned} g_0(a \mid w) &= Pr_0(\Delta_A = 1, \Delta_A A = a, \Delta_Y = 1 \mid W = w) \\ &= Pr_0(\Delta_A = 1 \mid W) Pr_0(\Delta_A A = a \mid \Delta_A = 1, W = w) \\ &\quad Pr_0(\Delta_Y = 1 \mid \Delta_A = 1, \Delta_A A = a, W = w). \end{aligned} \quad (11)$$

To demonstrate, we artificially introduce missing values to **A** and **Y** into the data from our running example.

```
DeltaA <- rbinom(n, 1, plogis(2 + W$W1))
DeltaY <- rbinom(n, 1, plogis(2 + W$W2 + A))
A[DeltaA == 0] <- NA
Y[DeltaY == 0] <- NA
```

The only required modification of the call to `drtmle()` is in how the PS estimation is performed. The options `glm_g` and `SL_g` are still used to fit generalized linear models or a super learner, respectively. However, the code allows for separate regression fits for each of the three components of the PS in equation (11). To perform separate generalized linear model fits for each of the three components, `glm_g` should be a named list with entries "DeltaA", "A", and "DeltaY". Respectively, these should provide a formula expression for the regression of Δ_A on W , A on W in observations with $\Delta_A = 1$, and of Δ_Y on W and A in observations with $\Delta_A = \Delta_Y = 1$ (if `stratify = FALSE`) or of Δ_Y on W to be fit in observations with $\Delta_A = \Delta_Y = 1$ and $A = a$ for each a specified in option `a.0` (if `stratify = TRUE`). If only a single formula is passed to `glm_g`, it is used for each of the three regressions. We illustrate a call to `drtmle()` with missing data and generalized linear model fits below.

```
glm_fit_wNAs <- drtmle(W = W, A = A, Y = Y, stratify = FALSE,
  glm_g = list(DeltaA = "W1 + W2", A = "W1 + W2",
    DeltaY = "W1 + W2 + A"),
  glm.Q = "W1 + W2*A", glm.Qr = "gn",
  glm_gr = "Qn", family = binomial())
glm_fit_wNAs
```

```
## $est
##
## 1 0.7353583
```

```
## 0 0.5213146
##
## $cov
##           1           0
## 1  1.836839e-03 -2.380141e-05
## 0 -2.380141e-05  4.917920e-03
```

It is possible to mix generalized linear models and super learning when fitting each of the components of the PS. In the example below, we use the wrapper function `SL.glm`, which fits a main terms logistic regression, to fit the regression of Δ_A on W ; use `SL.npreg` to fit the regression of A on W ; and use a super learner with library `SL.glm`, `SL.mean`, and `SL.earth` to fit the regression of Δ_Y on W and A .

```
mixed_fit_wNAs <- drtmle(W = W, A = A, Y = Y, stratify = FALSE,
                        SL_g = list(DeltaA = "SL.glm", A = "SL.npreg",
                                   DeltaY = c("SL.glm", "SL.mean", "SL.earth")),
                        glm_Q = "W1 + W2*A", glm_Qr = "gn",
                        glm_gr = "Qn", family = binomial())
```

We can also fit the PS regressions externally and pass in via `gn`, though this process is slightly more complicated than when no missing data are present. The basic idea is to fit one regression for (i) the indicator of missing A , (ii) A itself, and (iii) the indicator of the outcome being missing. The three PS's are then multiplied together and an appropriately ordered list is constructed as before. We illustrate with the following example.

```
# first regress indicator of missing A on W
fit_DeltaA <- glm(DeltaA ~ . , data = W, family = binomial())

# get estimated propensity for observing A
ps_DeltaA <- predict(fit_DeltaA, type = "response")

# now regress A on W | DeltaA = 1
fit_A <- glm(A[DeltaA == 1] ~ . , data = W[DeltaA == 1, , drop = FALSE],
            family = binomial())

# get estimated propensity for receiving A = 1
ps_A1 <- predict(fit_A, newdata = W, type = "response")
# propensity for receiving A = 0
ps_A0 <- 1 - ps_A1

# now regress DeltaY on A + W | DeltaA = 1. Here we are pooling over
# values of A (i.e., this is what drtmle does if stratify = FALSE).
# We could also fit two regressions, one of DeltaY ~ W | DeltaA = 1, A = 1
# and one of DeltaY ~ W | DeltaA = 1, A = 0 (this is what drtmle does if
# stratify = TRUE).
```

```

fit_DeltaY <- glm(DeltaY[DeltaA == 1] ~ . ,
                  data = data.frame(A = A, W)[DeltaA == 1, ],
                  family = binomial())

# get estimated propensity for observing outcome if A = 1
ps_DeltaY_A1 <- predict(fit_DeltaY, newdata = data.frame(A = 1, W),
                       type = "response")

# get estimated propensity for observing outcome if A = 0
ps_DeltaY_A0 <- predict(fit_DeltaY, newdata = data.frame(A = 0, W),
                       type = "response")

# now combine all results into a single propensity score
gn <- list(
  # propensity for DeltaA = 1, A = 1, DeltaY = 1
  ps_DeltaA * ps_A1 * ps_DeltaY_A1,
  # propensity for DeltaA = 1, A = 0, DeltaY = 1
  ps_DeltaA * ps_A0 * ps_DeltaY_A0
)

# pass in this gn to drtmle
out_fit_ps <- drtmle(W = W, A = A, Y = Y, stratify = FALSE,
                    # make sure a_0/gn are ordered properly!
                    gn = gn, a_0 = c(1, 0),
                    glm_Q = "W1 + W2*A", glm_Qr = "gn",
                    glm_gr = "Qn", family = binomial())

out_fit_ps

## $est
##
## 1 0.7353583
## 0 0.5213146
##
## $cov
##           1           0
## 1 1.836839e-03 -2.380141e-05
## 0 -2.380141e-05 4.917920e-03

```

Remark: In practice it is common to also have missingness in W . In this situation, there are varying assumptions under which one can still identify and estimate $E[Y(a)]$. The only native solution in `drtmle` currently is to augment W with columns of missingness indicators for each component of W . For example, if variable W_1 is observed, we could define a new indicator $\Delta_{W,1} = 1$ and $\Delta_{W,1} = 0$ otherwise. For observations with $\Delta_{W,1} = 0$, we could use some (possibly very simple) imputation strategy to fill in a value for W_1 . Candidate learners for the OR and PS may then include appropriate cross-products between

W_1 and $\Delta_{W,1}$. Other learners may opt to treat the imputed value of W_1 as the true value and ignore $\Delta_{W,1}$. Super learner can be used to help determine the modeling strategy that leads to the optimal model fit conditional on W_1 and $\Delta_{W,1}$. It is important to note that this approach is only valid for drawing causal inference under the assumption that $Y(a)$ is independent of A conditional on the partially observed variables in W and their missingness indicators. This assumption may not be plausible depending on the form of missingness of W . Other approaches for handling missing data, e.g., multiple imputation, could in theory be coupled with `drtmle`, though such applications have not been studied empirically and are not available natively in the `drtmle` package.

4.6 Multi-level treatments

In our examples so far, we have only considered binary treatments. However, `drtmle` is fully equipped to handle treatments with an arbitrary number of discrete values as well. Suppose that A assumes values in a set $\mathcal{A}$ of limited cardinality, the PS regression estimation is modified to ensure that

$$\sum_{a \in \mathcal{A}} g_n(a \mid w) = 1 \text{ for all } w .$$

If this condition is true, we say that the estimates of the PS are compatible. Many regression methodologies that have been developed work well with binary outcomes, but have not been extended to multi-level outcomes. To ensure that users of `drtmle` have access to the large number of binary regression techniques when estimating the PS, we have taken an alternative approach to estimation of the propensity for each level of the treatment. Specifically, rather than fitting a single multinomial-type regression, we fit a series of binomial regressions. As a concrete example, consider the case of no missing data and where $\mathcal{A} = \{0, 1, 2\}$. We can ensure compatible estimates of the PS by generating an estimate $g_n(0 \mid w)$ of $Pr_0(A = 0 \mid W = w)$ and $\tilde{g}_n(1 \mid w)$ of $Pr_0(A = 1 \mid A > 0, W = w)$. The latter estimate may be generated by regressing $I(A = 1)$ on W in observations with $A > 0$. Since the outcome is binary, this regression may be estimated using any technique suitable for binary outcomes. By rules of conditional probability, it holds that $g_0(1 \mid w) = Pr_0(A = 1 \mid A > 0, W = w)\{1 - Pr_0(A = 0 \mid W)\}$ and thus an estimate of $g_0(1 \mid w)$ can be computed as $g_n(1 \mid w) = \tilde{g}_n(1 \mid w)\{1 - g_n(0 \mid w)\}$. Finally, we let $g_n(2 \mid w) = 1 - g_n(0 \mid w) - g_n(1 \mid w)$, which ensures compatibility of the estimates for each level of the covariates. This approach generalizes to an arbitrary number of treatment levels. Below, we provide an illustration using simulated data with a treatment that has three levels. The true parameter values in this simulation are $\psi_0(0) = 0.62$, $\psi_0(1) = 0.72$, and $\psi_0(2) = 0.77$.

```
set.seed(1234)
n <- 200
W <- data.frame(W1 = runif(n), W2 = rbinom(n, 1, 0.5))
A <- rbinom(n, 2, plogis(W$W1 + W$W2))
Y <- rbinom(n, 1, plogis(W$W1 + W$W2*A))
```

The multi-level PS can still be estimated using any of the three techniques discussed previously. Here, we illustrate with generalized linear models.

```
glm_fit_multiA <- drtmle(W = W, A = A, Y = Y, stratify = FALSE,
                        glm.g = "W1 + W2", glm.Q = "W1 + W2*A",
                        glm.gr = "Qn", glm.Qr = "gn",
                        family = binomial(), a_0 = c(0,1,2))

glm_fit_multiA

## $est
##
## 0 0.4563335
## 1 0.7357243
## 2 0.7960642
##
## $cov
##           0           1           2
## 0  1.483804e-02 -3.223757e-05 -6.660173e-05
## 1 -3.223757e-05  2.178154e-03  6.376441e-05
## 2 -6.660173e-05  6.376441e-05  2.531909e-03
```

Above, we used the option `a_0` to specify the levels of the treatment at which we were interested in estimating marginal means. There is no need for `a_0` to contain all unique values of `A`. For example, certain levels of the treatment may not be of scientific interest, but nevertheless we would like to use these data to help estimate the OR and PS (by setting `stratify = FALSE`).

The confidence interval and testing procedures extend to multi-level treatments with minor modifications. We can still obtain a confidence interval and hypothesis test for each of the marginal means by not specifying a `contrast`.

```
ci(glm_fit_multiA)

## $drtmle
##      est    cil    ciu
## 0 0.456 0.218 0.695
## 1 0.736 0.644 0.827
## 2 0.796 0.697 0.895

wald.test(glm_fit_multiA, null = c(0.4, 0.5, 0.6))

## $drtmle
##              zstat  pval
## H0: E[Y(0)]=0.4 0.462 0.644
## H0: E[Y(1)]=0.5 5.051 0.000
## H0: E[Y(2)]=0.6 3.896 0.000
```

Alternatively, we can specify a `contrast` to estimate confidence intervals about the average treatment effect of $A = 1$ versus $A = 0$. Calls to `wald.test()` can also be made.

```
ci(glm_fit_multiA, contrast = c(-1, 1, 0))
```

```
## $drtmle
##               est    cil    ciu
## E[Y(1)]-E[Y(0)] 0.279 0.023 0.536
```

Similarly, we can compute confidence intervals comparing the average treatment effect of $A = 2$ versus $A = 0$.

```
ci(glm_fit_multiA, contrast = c(-1, 0, 1))
```

```
## $drtmle
##               est    cil    ciu
## E[Y(2)]-E[Y(0)] 0.34 0.08 0.599
```

User-specified contrasts are also available. For example, we can modify the `riskRatio` object we used previously for a two-level treatment to compute the risk ratio comparing $A = 1$ to $A = 0$ and comparing $A = 2$ to $A = 0$.

```
riskRatio_1v0 <- list(f = function(eff){ log(eff) },
                     f_inv = function(eff){ exp(eff) },
                     h = function(est){ est[2]/est[1] },
                     fh_grad = function(est){ c(1/est[2], -1/est[1], 0) })
ci(glm_fit_multiA, contrast = riskRatio_1v0)
```

```
## $drtmle
##               est cil    ciu
## user contrast 1.612 1.1 2.363
```

Similarly, we can compute confidence intervals for the risk ratio comparing the marginal mean for $A = 2$ to $A = 1$.

```
riskRatio_2v0 <- list(f = function(eff){ log(eff) },
                     f_inv = function(eff){ exp(eff) },
                     h = function(est){ est[3]/est[1] },
                     fh_grad = function(est){ c(0, -1/est[1], 1/est[3]) })
ci(glm_fit_multiA, contrast = riskRatio_2v0)
```

```
## $drtmle
##               est    cil    ciu
## user contrast 1.744 1.382 2.202
```

4.7 Cross-validated regression estimates

To avoid overfitting initial regressions, one can use cross-validated estimates of the nuisance regressions. Theoretically, the use of cross-validated regression weakens entropy conditions on nuisance parameter estimators necessary to achieve asymptotic normality (Klaassen 1987; Bickel et al. 1993; Zheng and van der Laan 2010). This is true of the regular TML and AIPTW estimators, as well as of the TML estimator with doubly robust inference. The `drtmle()` function implements cross-validated estimation of the nuisance regression parameters through the `cvFolds` option, as shown in the code below.

```
cv_sl_fit <- drtmle(W = W, A = A, Y = Y, family = binomial(),
  SL_g = c("SL.glm", "SL.glm.interaction", "SL.earth"),
  SL_Q = c("SL.glm", "SL.glm.interaction", "SL.earth"),
  SL_gr = c("SL.glm", "SL.mean"),
  SL_Qr = c("SL.glm", "SL.mean"),
  stratify = FALSE, cvFolds = 2, a_0 = c(0, 1, 2))
```

Cross-validation may significantly increase the computation time necessary for running `drtmle()`. Parallelized nuisance regression fitting is available by setting the option `use_future = TRUE`. Since, internally, all computations are carried out using futures (with the API provided by the `future` R package), setting this option amounts only to parallelizing with futures (Bengtsson 2017a, 2021) via `future.apply::lapply()`; when `use_future = FALSE`, base `lapply()` is simply used instead. Together, setting these options results in parallelization of the estimation of the OR, PS, and reduced-dimension regressions.

If no adjustments are made prior to invoking the `drtmle()` function, futures are computed sequentially, as per the defaults of the `future` package. It is left to the user to specify appropriate system-specific future backends that may better suit both their analytical problem and computing environment (e.g., using the `future.batchtools` package (Bengtsson 2017b) or even simply setting a parallelized `future::plan()`). Below, we demonstrate registration using an ad-hoc cluster on the local system. The choice of backend is flexible, and submission of the job to a variety of schedulers (e.g., SLURM, TORQUE) is also permitted. Interested readers are invited to consult the documentation of the `future.batchtools` package for further information.

```
# set up parallel backend via future.batchtools
library(future.batchtools)
library(parallel)
cl <- makeCluster(2, type = "SOCK")
plan(cluster, workers = cl)
clusterEvalQ(cl, library("SuperLearner"))

# run drtmle() in parallel using the registered backend via use_future = TRUE
pcv_sl_fit <- drtmle(W = W, A = A, Y = Y, family = binomial(),
  SL_g = c("SL.glm", "SL.glm.interaction", "SL.earth"),
  SL_Q = c("SL.glm", "SL.glm.interaction", "SL.earth"),
  SL_gr = c("SL.glm", "SL.earth", "SL.mean"),
```

```
SL_Qr = c("SL.glm", "SL.earth", "SL.mean"),
stratify = FALSE, a_0 = c(0,1,2),
cvFolds = 2, use_future = TRUE)
```

5. Discussion

The `drtmle` package was designed to provide a user-friendly implementation of the TMLE algorithm that facilitates doubly robust inference about marginal means and average treatment effects. While the theory underlying doubly robust inference is complex, the implementation of the methods only requires users to provide the data and specify how to estimate nuisance regressions. Though estimation of the OR and PS is familiar to those with backgrounds in causal inference-related estimation, the reduced-dimension regressions may appear strange. Indeed, it is difficult to provide intuition for these parameters and more difficult still to determine what scientific background knowledge might guide users in how to estimate these regression parameters. Therefore, we recommended estimating these quantities adaptively, e.g., with the super learner algorithm. The super learner library should contain several relatively smooth estimators such as `SL.mean`, `SL.glm`, and `SL.gam`. In particular, including `SL.mean` (an unadjusted generalized linear model) may be important due to the fact that, when the OR and PS are consistently estimated, the reduced-dimension regressions are identically equal to zero. Thus, this unadjusted regression estimator may do a good job estimating the reduced-dimension regressions in situations where the OR and PS are estimated well. In addition to relatively smooth estimators, we recommend additionally including several adaptive estimators such as `SL.npreg` or `SL.earth`.

Planned extensions of the `drtmle` package include extensions to longitudinal settings involving treatments at multiple time points and inclusion of the collaborative targeted maximum likelihood estimator (C-TMLE) of M. van der Laan (2014) that also facilitates collaborative doubly robust inference. The `drtmle` R package is publicly available on the Comprehensive R Archive Network (CRAN) at <https://CRAN.R-project.org/package=drtmle>, and the open source development repository is available on the GitHub platform for collaborative programming at <https://github.com/benkese/drtmle>; the package's documentation (automatically built via the `pkgdown` R package) is hosted via GitHub Pages at <https://benkeser.github.io/drtmle/>.

6. Appendix

6.1 Arbitrary user-specified regressions

The `drtmle` package also allows users to pass in their own estimated OR and PS via the `Qn` and `gn` options, respectively. If `Qn` is specified by the user, `drtmle()` will ignore the nuisance parameter estimation routines specified by `SL.Q` and `glm.Q` – similarly for `gn` and `SL.g`, `glm.g`. As noted in the package documentation, there is a specific format necessary for inputting `Qn` and `gn`. For `Qn`, the entries in the list should correspond to the OR evaluated at `A` and the observed values of `W`, with order determined by the input to `a_0`. For example, if `a_0 = c(0, 1)` then `Qn[[1]]` should be the OR at $A = 0$ and `Qn[[2]]` should be the outcome regression at $A = 1$. The example below illustrates this concept.

```

# use simplified data
set.seed(1234)
n <- 200
W <- data.frame(W1 = runif(n), W2 = rbinom(n, 1, 0.5))
A <- rbinom(n, 1, plogis(W$W1 + W$W2))
Y <- rbinom(n, 1, plogis(W$W1 + W$W2*A))

# fit a GLM for outcome regression outside of drtmle
out_glm_fit <- glm(Y ~ ., data = data.frame(A = A, W), family = binomial())

# generate Qn list
Qn10 <- list(
  # first entry is predicted values setting A = 1
  predict(out_glm_fit, newdata = data.frame(A = 1, W), type = "response"),
  # second entry is predicted values setting A = 0
  predict(out_glm_fit, newdata = data.frame(A = 0, W), type = "response")
)

# pass this list to drtmle to avoid internal estimation of
# outcome regression (note propensity score and reduced-dimension
# regressions are still estimated internally)
out_fit1 <- drtmle(W = W, A = A, Y = Y, family = binomial(),
  glm_g = ".", glm_Qr = "gn", glm_gr = "Qn", Qn = Qn10,
  # crucial to set a_0 to match Qn's construction!
  a_0 = c(1,0))

out_fit1

## $est
##
## 1 0.7116610
## 0 0.5376892
##
## $cov
##           1           0
## 1 1.726990e-03 1.428573e-06
## 0 1.428573e-06 4.183006e-03

# to be completely thorough let's re-order Qn10 and re-run
Qn01 <- list(Qn10[[2]], Qn10[[1]])

out_fit2 <- drtmle(W = W, A = A, Y = Y, family = binomial(),
  glm_g = ".", glm_Qr = "gn", glm_gr = "Qn",
  # use re-ordered Qn list
  Qn = Qn01,
  # a_0 has to be reordered to match Qn01!

```

```

      a_0 = c(0,1))
# should be the same as out_fit1, but re-ordered
out_fit2

```

```

## $est
##
## 0 0.5376892
## 1 0.7116610
##
## $cov
##           0           1
## 0 4.183006e-03 1.428573e-06
## 1 1.428573e-06 1.726990e-03

```

Similarly, for `gn` we need to pass in properly ordered lists. For example, if `a_0 = c(0, 1)` then `gn[[1]]` should be the PS for receiving $A = 0$ and `gn[[2]]` should be the PS for receiving $A = 1$. The example below illustrates this concept.

```

# fit a GLM for propensity score outside of drtmle
out_glm_fit <- glm(A ~ . , data = data.frame(W), family = binomial())

# get P(A = 1 | W) by calling predict
gn1W <- predict(out_glm_fit, newdata = data.frame(W), type = "response")

# generate gn list
gn10 <- list(
  # first entry is P(A = 1 | W)
  gn1W,
  # second entry is P(A = 0 | W) = 1 - P(A = 1 | W)
  1 - gn1W
)

# pass this list to drtmle to avoid internal estimation of
# propensity score (note reduced-dimension regressions are
# still estimated internally)
out_fit3 <- drtmle(W = W, A = A, Y = Y, family = binomial(),
  glm_Qr = "gn", glm_gr = "Qn",
  Qn = gn10, gn = gn10,
  # crucial to set a_0 to match Qn and gn's construction!
  a_0 = c(1,0))
out_fit3

## $est
##
## 1 0.7116610

```

```
## 0 0.5376892
##
## $cov
##           1           0
## 1 1.726990e-03 1.428573e-06
## 0 1.428573e-06 4.183006e-03

# to be completely thorough let's re-order gn10 and re-run
gn01 <- list(gn10[[2]], gn10[[1]])

out_fit4 <- drtmle(W = W, A = A, Y = Y, family = binomial(),
                  glm_Qr = "gn", glm_gr = "Qn",
                  # use re-ordered Qn/gn list
                  Qn = Qn01, gn = gn01,
                  # a_0 has to be reordered to match Qn01!
                  a_0 = c(0,1))
# should be the same as out_fit3, but re-ordered
out_fit4

## $est
##
## 0 0.5376892
## 1 0.7116610
##
## $cov
##           0           1
## 0 4.183006e-03 1.428573e-06
## 1 1.428573e-06 1.726990e-03
```

This flexibility allows users to leverage any regression technique to externally obtain initial estimates of nuisance parameters. The key step is making sure that the ordering of `Qn` and `gn` is consistent with `a_0`.

6.2 Additional methods included in drtmle

6.2.1 INFERENCE FOR IPTW USING ADAPTIVE PS ESTIMATORS

Doubly robust estimators of marginal means are appealing in their robustness and efficiency properties; however, researchers may prefer the IPTW estimator for its intuitive derivation and implementation. Heretofore, inference for IPTW estimators precluded the use of adaptive estimators of the PS, yet M. van der Laan (2014) proposed modified IPTW estimators that allow for adaptive PS estimation by incorporating an additional step to update the initial PS estimates. Like those presented above, this estimator is based on an additional regression parameter, $\tilde{Q}_{r,0n}(a, w) := E_0\{Y \mid A = a, g_n(W) = g_n(w)\}$, that is, the regression of the outcome Y on the estimated propensity score amongst observations with $A = a$. The author showed that if $g_n^*(a \mid w)$, an estimator of the propensity for treatment $A = a$,

satisfied that

$$\frac{1}{n} \sum_{i=1}^n \frac{\tilde{Q}_{r,0}(a, W_i)}{g_n(a | W = W_i)} \{I(A_i = a) - g_n(a | W = W_i)\} = 0 ,$$

then the IPTW estimator based on g_n^* would asymptotically attain a normal limiting distribution with estimable variance. In fact, M. van der Laan (2014) proposed a TMLE-type algorithm that mapped an initial PS estimate into an estimate that satisfies this key equation and variance estimators that can be used to generate Wald-style confidence intervals and hypothesis tests.

An estimator with equivalent asymptotic properties based on a PS estimate, $g_n(a | w)$, can be computed as

$$\psi_{n,\text{IPTW}}(a) = \psi_{n,\text{IPTW}}(a) - \frac{1}{n} \sum_{i=1}^n \frac{\tilde{Q}_{r,0}(a, W_i)}{g_n(a | W = W_i)} \{I(A_i = a) - g_n(a | W = W_i)\} .$$

Both this estimator and the TMLE-based IPTW estimator above are implemented in `drtmle`. Alternative approaches for developing IPTW estimators that are asymptotically normal and attain the nonparametric efficiency bound have been investigated by Ertefaie, Hejazi, and van der Laan (2022) and Hejazi et al. (2022), who relied on undersmoothing-based corrections rather than the TMLE-type corrections of M. van der Laan (2014).

Implementation of inference for adaptive IPTW The IPTW estimators based on adaptive estimation of the PS are implemented in the `adaptive_iptw()` function. The `adaptive_iptw()` function shares many options with `drtmle()`; however, as the IPTW estimator does not rely on an estimate of the OR, options related to the OR are omitted.

```
npreg_iptw_multiA <- adaptive_iptw(W = W, A = A, Y = Y, stratify = FALSE,
                                   SL_g = "SL.npreg", SL_Qr = "SL.npreg",
                                   family = binomial(), a_0 = c(0, 1))
npreg_iptw_multiA
```

```
## $est
##
## 0 0.5331529
## 1 0.7160554
##
## $cov
##           0           1
## 0 3.908130e-03 -2.470024e-06
## 1 -2.470024e-06 1.677193e-03
```

The `"adaptive_iptw"` S3 class contains identical methods for computing confidence intervals and Wald-style tests as the `"drtmle"` class, thus readers are referred back to the section on confidence intervals and hypothesis tests for reminders on the various options for these procedures. By default, these methods are implemented for the `iptw_tmle()`

estimator of M. van der Laan (2014). We conjecture that this estimator will have improved finite-sample behavior relative to `iptw_os()` (the alternative estimator described above), though further study in numerical experiments is warranted.

As with `drtmle()`, the `adaptive_iptw()` function allows missing data in `A` and `Y`. Similarly, `adaptive_iptw()` also allows for parallelized, cross-validated estimation of the regression parameters via the `cvFolds` and `parallel` options. As with the estimators implemented by `drtmle()`, cross-validation allows for valid inference under weaker assumptions.

6.2.2 OUTCOME-ADAPTIVE PROPENSITY SCORE ESTIMATION

Ju, Benkeser, and van der Laan (2020) have recently proposed estimators of the average treatment effect to be used in settings where this effect is weakly identifiable, as evidenced by estimated propensity scores of small magnitude. In these settings, doubly robust estimators may exhibit non-robust behavior in small samples. The proposed solution is to target an alternative quantity in the propensity score estimation step. If we are interested in estimating $\psi_0(a)$ for $a = 0, 1$, we can estimate $Pr_0(A = a \mid \bar{Q}_0(1, W), \bar{Q}_0(0, W))$, which describes the conditional probability of receiving treatment $A = a$ given the value of the two outcome regressions. For this reason, we call this quantity an *outcome-adaptive propensity score*. We have shown that all of the usual asymptotics for TMLE carry through when an estimator of this quantity is substituted for the true propensity score. The resultant TML estimator is *super-efficient*, meaning that it will generally have greater asymptotic efficiency than a standard TML estimator (in fact, greater asymptotic efficiency than is ordinarily available to any regular asymptotically linear estimator). Simulation experiments indicate that this approach also delivers more stable behavior in settings where estimated propensity scores become very small. However, the estimator is not doubly robust – the asymptotics rely on the outcome regression being consistently estimated at $n^{-1/4}$ rate. The highly adaptive lasso estimator (David Benkeser and van der Laan 2016; M. J. van der Laan 2017; Bibaut and van der Laan 2019), implemented in the open source `hal9001` R package (Hejazi, Coyle, and van der Laan 2020; Coyle et al. 2022), can attain such a convergence rate without sacrificing flexible estimation; nevertheless, as a consequence of this convergence rate requirement, this estimator trades off robustness against achieving greater stability and efficiency. See Ju, Benkeser, and van der Laan (2020) for further discussion.

Implementation of outcome-adaptive propensity score If the outcome-adaptive PS is desired, the user may specify this via the `adapt_g` option. In this case, the additional targeting steps needed for doubly robust inference are “turned off” (i.e., `guard = NULL`), since, as discussed above, this estimator is *not* doubly robust. Note that when `adapt_g = TRUE`, the data used for PS estimation will be a `data.frame` with column names `QaW` where `a` takes all the values specified in `a_0`. For example if `a_0 = c(0, 1)`, then the PS estimation data frame will have two columns called `Q0W` and `Q1W`. This may be relevant for users writing their own `SuperLearner` wrapper functions or those who make use of the `glm_g` setting.

```
# outcome adaptive propensity score fit using glms
adaptg_fit <- drtmle(W = W, A = A, Y = Y, family = binomial(),
  glm_Q = ".^2", glm_g = "Q1W + Q0W",
  adapt_g = TRUE)
```

```
# outcome adaptive propensity score fit using super learner
adaptg_sl_fit <- drtmle(W = W, A = A, Y = Y, family = binomial(),
  SL_Q = c("SL.glm", "SL.mean"),
  SL_g = c("SL.glm", "SL.mean"),
  adapt_g = TRUE)
```

Acknowledgements

NSH grateful acknowledges the support of the National Science Foundation (award no. DMS 2102840).

References

- Bach, Philipp, Victor Chernozhukov, Malte S Kurz, and Martin Spindler. 2021. “DoubleML – an Object-Oriented Implementation of Double Machine Learning in R.” *arXiv Preprint arXiv:2103.09603*.
- . 2022. “DoubleML – an Object-Oriented Implementation of Double Machine Learning in Python.” *Journal of Machine Learning Research* 23.
- Bengtsson, Henrik. 2017a. *future: A Future API for R*. <https://cran.r-project.org/web/packages/future>. <https://github.com/HenrikBengtsson/future>.
- . 2017b. *future.batchtools: A Future API for Parallel and Distributed Processing Using “Batchtools”*. <https://cran.r-project.org/web/packages/future.batchtools>. <https://github.com/HenrikBengtsson/future.batchtools>.
- . 2021. “A Unifying Framework for Parallel and Distributed Processing in R Using Futures.” *The R Journal* 13 (2): 273–91. <https://doi.org/10.32614/RJ-2021-048>.
- Benkeser, David, and Mark J van der Laan. 2016. “The Highly Adaptive Lasso Estimator.” In *2016 IEEE International Conference on Data Science and Advanced Analytics (DSAA)*, 689–96. IEEE.
- Benkeser, D, M Carone, M van der Laan, and P Gilbert. 2017. “Doubly-Robust Nonparametric Inference on the Average Treatment Effect.” *Biometrika*.
- Bibaut, Aurélien F, and Mark J van der Laan. 2019. “Fast Rates for Empirical Risk Minimization over Càdlàg Functions with Bounded Sectional Variation Norm.” *arXiv Preprint arXiv:1907.09244*. <https://arxiv.org/abs/1907.09244>.
- Bickel, Peter J, Chris AJ Klaassen, YA’Acov Ritov, and Jon A Wellner. 1993. *Efficient and Adaptive Estimation for Semiparametric Models*. Johns Hopkins University Press Baltimore.
- Breiman, Leo. 1996. “Stacked Regressions.” *Machine Learning* 24 (1): 49–64. <https://doi.org/10.1023/A:1018046112532>.
- Chernozhukov, Victor, Denis Chetverikov, Mert Demirer, Esther Duflo, Christian Hansen, and Whitney Newey. 2017. “Double/Debiased/Neyman Machine Learning of Treatment Effects.” *American Economic Review* 107 (5): 261–65.
- Chernozhukov, Victor, Denis Chetverikov, Mert Demirer, Esther Duflo, Christian Hansen, Whitney Newey, and James M Robins. 2018. “Double/Debiased Machine Learning

- for Treatment and Structural Parameters.” *The Econometrics Journal* 21 (1): C1–68. <https://doi.org/10.1111/ectj.12097>.
- Coyle, Jeremy R. 2021. *tmle3: The Extensible TMLE Framework*. <https://github.com/tlverse/tmle3>. <https://doi.org/10.5281/zenodo.4603358>.
- Coyle, Jeremy R, Nima S Hejazi, Rachael V Phillips, Lars WP van der Laan, and Mark J van der Laan. 2022. *hal9001: The Scalable Highly Adaptive Lasso*. <https://doi.org/10.5281/zenodo.3558313>.
- Dudoit, Sandrine, and Mark J van der Laan. 2005. “Asymptotics of Cross-Validated Risk Estimation in Estimator Selection and Performance Assessment.” *Statistical Methodology* 2 (2): 131–54.
- Ertefaie, Ashkan, Nima S Hejazi, and Mark J van der Laan. 2022. “Nonparametric Inverse Probability Weighted Estimators Based on the Highly Adaptive Lasso.” *Biometrics* (just accepted). <https://arxiv.org/abs/2005.11303>.
- Gruber, Susan, and Mark J van der Laan. 2012. “tmle: An R Package for Targeted Maximum Likelihood Estimation.” *Journal of Statistical Software* 51: 1–35.
- Hayfield, Tristen, and Jeffrey Racine. 2008. “Nonparametric Econometrics: The np Package.” *Journal of Statistical Software* 27 (5). <https://www.jstatsoft.org/v27/i05/>.
- Hejazi, Nima S, David Benkeser, Ivan Diaz, and Mark J van der Laan. 2022. “Efficient Estimation of Modified Treatment Policy Effects Based on the Generalized Propensity Score.” <https://arxiv.org/abs/2205.05777>.
- Hejazi, Nima S, Jeremy R Coyle, and Mark J van der Laan. 2020. “hal9001: Scalable Highly Adaptive Lasso Regression in R.” *Journal of Open Source Software* 5 (53): 2526. <https://doi.org/10.21105/joss.02526>.
- Horvitz, D, and D Thompson. 1952. “A Generalization of Sampling Without Replacement from a Finite Universe.” *Journal of the American Statistical Association* 47 (260): 663–85. <https://doi.org/10.1080/01621459.1952.10483446>.
- Ju, Cheng, David Benkeser, and Mark J van der Laan. 2020. “Robust Inference on the Average Treatment Effect Using the Outcome Highly Adaptive Lasso.” *Biometrics* 76 (1): 109–18.
- Klaassen, Chris AJ. 1987. “Consistent Estimation of the Influence Function of Locally Asymptotically Linear Estimators.” *The Annals of Statistics*, 1548–62.
- Lendle, Samuel D, Joshua Schwab, Maya L Petersen, and Mark J van der Laan. 2017. “Itmle: An R Package Implementing Targeted Minimum Loss-Based Estimation for Longitudinal Data.” *Journal of Statistical Software* 81: 1–21.
- Milborrow, S et al. 2018. *earth: Multivariate Adaptive Regression Splines*. <https://CRAN.R-project.org/package=earth>.
- Polley, E, E LeDell, C Kennedy, S Lendle, and M van der Laan. 2017. *SuperLearner: Super Learner Prediction*. <https://CRAN.R-project.org/package=SuperLearner>.
- Porter, K, S Gruber, M van der Laan, and J Sekhon. 2011. “The Relative Performance of Targeted Maximum Likelihood Estimators.” *International Journal of Biostatistics* 7 (1). <https://doi.org/10.2202/1557-4679.1308>.
- R Core Team. 2022. *R: A Language and Environment for Statistical Computing*. Vienna, Austria: R Foundation for Statistical Computing. <https://www.R-project.org/>.
- Robins, J. 1986. “A New Approach to Causal Inference in Mortality Studies with a Sustained Exposure Period — Application to Control of the Healthy Worker Survivor

- Effect.” *Mathematical Modelling* 7 (9–12): 1393–1512. [https://doi.org/10.1016/0270-0255\(86\)90088-6](https://doi.org/10.1016/0270-0255(86)90088-6).
- Robins, J, M Hernan, and B Brumback. 2000. “Marginal Structural Models and Causal Inference in Epidemiology.” *Epidemiology* 11 (5): 550–60. <https://doi.org/10.1097/00001648-200009000-00011>.
- Robins, J, A Rotnitzky, and L Zhao. 1994. “Estimation of Regression Coefficients When Some Regressors Are Not Always Observed.” *Journal of the American Statistical Association* 89 (427): 846–66. <https://doi.org/10.1080/01621459.1994.10476818>.
- van der Laan, M. 2014. “Targeted Estimation of Nuisance Parameters to Obtain Valid Statistical Inference.” *International Journal of Biostatistics* 10 (1): 29–57. <https://doi.org/10.1515/ijb-2012-0038>.
- van der Laan, Mark J. 2017. “A Generally Efficient Targeted Minimum Loss Based Estimator Based on the Highly Adaptive Lasso.” *The International Journal of Biostatistics* 13 (2).
- van der Laan, Mark J, David Benkeser, and Weixin Cai. 2019. “Causal Inference Based on Undersmoothing the Highly Adaptive Lasso.” *Proceedings of the Association for the Advancement of Artificial Intelligence Spring Symposium*. <https://why19.causalai.net/papers/laan-why19.pdf>.
- van der Laan, Mark J, Jeremy R Coyle, Nima S Hejazi, Ivana Malenica, Rachael V Phillips, and Alan E Hubbard. 2023. *Targeted Learning in R: Causal Data Science with the tlverse Software Ecosystem*. CRC Press. <https://tlverse.org/tlverse-handbook>.
- van der Laan, Mark J, Eric Polley, and Alan E Hubbard. 2007. “Super Learner.” *Statistical Applications in Genetics and Molecular Biology* 6 (1). <https://doi.org/10.2202/1544-6115.1309>.
- van der Laan, Mark J, and Sherri Rose. 2011. *Targeted Learning: Causal Inference for Observational and Experimental Data*. Springer New York. <https://doi.org/10.1007/978-1-4419-9782-1>.
- van der Laan, Mark J, and Dan Rubin. 2006. “Targeted Maximum Likelihood Learning.” *International Journal of Biostatistics* 2 (1). <https://doi.org/10.2202/1557-4679.1043>.
- van der Vaart, Aad W, Sandrine Dudoit, and Mark J van der Laan. 2006. “Oracle Inequalities for Multi-Fold Cross-Validation.” *Statistics and Decisions* 24 (3): 351–71. <https://doi.org/10.1524/stnd.2006.24.3.351>.
- Wolpert, David. 1992. “Stacked Generalization.” *Neural Networks* 5 (2): 241–59. [https://doi.org/10.1016/S0893-6080\(05\)80023-1](https://doi.org/10.1016/S0893-6080(05)80023-1).
- Zheng, Wenjing, and Mark J van der Laan. 2010. “Asymptotic Theory for Cross-Validated Targeted Maximum Likelihood Estimation.” *U.C. Berkeley Division of Biostatistics Working Paper Series*. <https://biostats.bepress.com/ucbbiostat/paper273/>.